

VERIQO Security FAQ

Written for a security reviewer evaluating VERIQO for a pilot. Answers are specific and cite the mechanism. Where the answer is "no" or "not yet," it says so — a security questionnaire answered optimistically is worse than useless.

The precise readiness position is in `docs/VERIQO_READINESS_TIER_FRAMEWORK.md`. Summary: VERIQO is **not production-qualified** and has **not** had an independent penetration test.

Data

Q: What customer data does VERIQO store? Evidence *metadata* — identifiers, a caller-supplied SHA-256, a URI pointing at the customer's own storage, filename, media type, byte size, collector, source, domain metadata — plus decisions, authorizations, and the audit ledger. **VERIQO does not store evidence bytes.** Documents remain in the customer's custody.

Q: So what happens if VERIQO is breached? An attacker obtains metadata and hashes, not the underlying documents. That metadata can itself be sensitive (claim IDs, vessel identities, party identifiers, filenames), so it is not "nothing" — but the documents are not there to take.

Q: Is data encrypted at rest? Not by the application. The durable store writes WAL segment files to local disk unencrypted. Disk-level encryption is a deployment concern (encrypted volumes / encrypted filesystems), and must be configured by whoever operates the deployment. This is a real, stated gap, not an implicit guarantee.

Q: In transit? TLS, including mutual TLS, is supported and configured at the gateway (`security.LoadServerTLSConfig` / `LoadMutualTLSConfig`, selected by the gateway's own flags). TLS is not enabled by default — a deployment without it configured serves plaintext HTTP, so verify this for your deployment specifically.

Authentication and authorization

Q: How do callers authenticate? A composed middleware stack, outermost first: Audit → JWT → RBAC → API key. Each layer is independently enabled by configuration; a deployment with none configured is unauthenticated.

Q: Do you support OIDC / our IdP? No. JWT verification is of locally HS256-signed tokens issued by the deployment, not a third-party IdP's tokens. Adding OIDC is an additional verification path, not a redesign — but it does not exist today.

Q: Can one tenant read another's data? No, on two independent levels. Records are keyed by `tenantID + "/" + ID`, so a cross-tenant lookup fails at "not found" before any ownership check runs — the isolation is structural, not a comparison someone could forget to write. Adversarial tests (`TestTenantAISolationFromTenantB`, 8 sub-tests, plus an HTTP-level equivalent) prove tenant A cannot read, modify, or replay tenant B's case.

Q: Can a caller just claim to be a different tenant? No, when JWT authentication is configured. The effective tenant is derived from the verified JWT subject through a tenant-membership registry; a request naming a tenant the subject is not granted is refused with 403 even with an otherwise valid token. If the membership registry is

absent while JWT is configured, every authenticated request is refused — it fails closed rather than falling back to trusting the client.

If **no** JWT layer is configured, the supplied tenant is used as-is. That mode is for single-tenant or trusted-network deployments; do not use it for multi-tenant production.

Q: Is authorization tenant-scoped? Partially. Data access is. RBAC is not: the role table is global (one role-to-path-prefix map per deployment), so roles are not yet per-tenant. Stated as an open item.

Integrity and cryptography

Q: What actually guarantees integrity? Evidence manifests are hashed over canonicalized (JCS) content; the custody chain is hash-linked; the audit ledger is hash-chained with each record binding the previous one. Any of these can be re-verified independently at any time — including by the standalone verifier, out of process.

Q: Are evidence and dossiers signed? Yes, when signing is enabled: real Ed25519 signatures over the evidence manifest hash and over the dossier package hash, with a real key lifecycle (PENDING → ACTIVE → RETIRED → REVOKED). Revocation is retroactive — signatures made while a key was active fail verification once it is revoked.

Signing is **off by default**. Unsigned records are visibly unsigned (the signature field is absent), never presented as if signed.

Q: Where do signing keys live? In process memory or a local file, via the key-provider abstraction. **Not in an HSM or cloud KMS.** An AWS KMS adapter shape exists but is not wired to a live KMS tenancy (blocked on procurement). This is adequate for a pilot's trust model and is **not** acceptable permanent production key custody.

Q: Is a signature part of what it signs? No. Hashing is always hash-then-sign: the dossier's package hash is computed with both the hash field and the signature field zeroed, so verification recomputes over exactly the same bytes. A signature can never be circularly included in its own digest.

Independent verification

Q: Can we verify a dossier without trusting your server? Yes — that is the design intent. `veriqo-commercial-verify` is a separate compiled binary that reads only the exported `.zip` and, optionally, a trusted-key registry you obtained through your own channel. It makes no network calls and shares no memory with any running VERIQO service.

Q: Does it really check signatures, or does it skip them? It performs real Ed25519 signature and key-state verification **when you supply a trusted-key registry**. Without one, signature and key-state checks report `SKIP` with the reason stated — never a false `PASS`. The distinction matters: the capability is real; whether a given run exercises it depends on whether your organization has established key distribution.

Note that the HTTP convenience route `POST /v1/packages/verify` always verifies with no registry, so it always reports `SKIP` for signatures. The CLI is the verifier of record.

Q: What does it check besides signatures? Package hash, manifest integrity, raw evidence hashes, custody chain, Merkle proof, lineage (cross-referencing the dossier's claimed decision and authorization hashes against the independently-parsed ledger), and the ledger's own hash chain from genesis.

Availability and operations

Q: Does data survive a crash? Yes. Every mutating call is appended to a write-ahead log (fsync, CRC, defect-classified recovery) before the caller is told it succeeded. On restart the store replays those inputs through its own real logic and reconstructs byte-identical state. A crash mid-write produces a truncated, *reported* tail — never silent data loss beyond the single unconfirmed record. Tested including a simulated torn write and a live kill-and-restart against the compiled binary.

Q: Backup and restore? Backup copies the durable log after fsyncing; restore replays it through the identical code path as ordinary crash recovery — a backup is deliberately not a separate, untested format. Round trip is tested. **No operational drill against a live deployment has been performed**, and there is no automated backup scheduler; that is deployment work.

Q: Health probes? GET /livez (dependency-free liveness) and GET /readyz (dependency-aware; reports 503 when the durable store is not fit to serve). Both are exempt from the auth stack.

Q: DoS protections? Request bodies are bounded (1 MiB JSON, 64 MiB package upload). There is **no** application-level rate limiting, connection throttling, or quota enforcement — that belongs to a reverse proxy or API gateway in front of the deployment.

Logging and audit

Q: What is logged? Method, path, status, and latency per request, with the request path explicitly escaped against log injection. Business-level actions (decisions, authorizations, executions) go to the hash-chained audit ledger, which is tamper-evident: altering a record breaks the chain and VerifyChain detects it.

Q: Could an operator quietly alter the ledger? Not without detection. The chain binds each record to its predecessor's hash, and verification recomputes from genesis. An operator with disk access could delete or truncate — which is detectable as a broken or short chain, not as a plausible alternative history.

Q: Do you have distributed tracing or alerting? No to both. Metrics (GET /v1/metrics) and logs are real; a customer's own monitoring stack must scrape and alert.

Process and assurance

Q: Has VERIQO been penetration tested? No. The engineering harness for qualification exists and passes; the external engagement has not happened. Reported as READY_FOR_REAL_QUALIFICATION, which is explicitly not QUALIFIED.

Q: Independent security audit? No.

Q: Do you have an incident response procedure? Yes, as of this round — see docs/customer/VERIQO_INCIDENT_RESPONSE_PROCEDURE.md. It has not yet been exercised in a real incident or a tabletop.

Q: Vulnerability scanning / supply chain? The codebase has zero external Go dependencies for the gateway and core paths (standard library only), which materially reduces supply-chain surface. Static analysis (gosec, staticcheck) and SBOM generation are part of the qualification harness. A live vulnerability-feed integration (govulncheck against a current database) is not wired in.

Q: What is your disclosure process? Report suspected vulnerabilities through your pilot contact. During a pilot there is no published security.txt or bug-bounty programme.

The honest summary

VERIQO's *trust* properties — integrity, custody, grounding, independent verifiability, tamper-evident audit, durability, tenant isolation — are real, tested, and adversarially proven.

VERIQO's *enterprise security operations* properties — OIDC, HSM-backed key custody, rate limiting, tracing, alerting, encryption at rest, an executed penetration test, an independent audit — are partly absent and named above. A security reviewer should treat this as a system whose core is well-built and whose production hardening is incomplete, and gate a paid pilot on the specific items that matter to your risk posture.